

占城大师开发方式与项目结构

PDF 生成日期: 2026-07-10 | 源文件: docs/DEVELOPMENT_WORKFLOW.md

本文档是 zhanchengdashi 的日常开发入口, 也是公共《通用游戏开发流程文档》在本项目里的适配层。

公共流程上游:

- C:\Users\76398\Documents\Codex\2026-07-03\codex-game-studio-default\outputs\codex-game-studio-general-game-development-process.md
- 当前对齐版本: 公共流程 v1.9, 日期 2026-07-10。除 v1.6 已同步的 2D-first 与专业美术生产线外, 本次重点同步 Figma/FigJam UI/UE 正式源稿、UI 效果图工程交付、design tokens、组件状态矩阵、响应式/安全区、截图对照和 UI QA Gate。

使用方式: 公共流程负责角色路由、生命周期、专项流程、任务卡和 Definition of Done; 本项目文档负责把这些规则落到 zhanchengdashi 的目录结构、Godot/GDScript、配置表、验证脚本和 GitHub 同步上。后续开发必须优先遵循公共流程, 再按本项目约定执行。

0. 当前项目流程确认

- 上游流程: 使用公共 codex-game-studio-general-game-development-process.md。
- 项目适配: 使用本文档 docs/DEVELOPMENT_WORKFLOW.md。
- 项目阶段: 当前属于 Prototype / Vertical Slice 之间, 已经有 Godot 可运行原型、卡牌/抽卡/战斗/编组闭环和配置表基础。
- 默认引擎角色: 公共流程中的 Engine Specialist 在本项目映射为 Godot Specialist。
- 默认语言角色: 公共流程中的 Language Specialist 在本项目映射为 GDScript Specialist。
- 默认维度: 本项目是 2D-first。除非用户明确改为 3D/2.5D, 否则所有效果图、UI 皮肤、角色、场景、地图和 FX 都按 2D 生产线处理。
- 默认美术表现角色: 美术方向、效果图、UI 视觉、角色/场景/道具/地图/FX 资产默认启用 Art Director -> Visual Development Artist -> Concept Artist / Environment Artist / UI Artist -> 2D Animation Specialist -> Sprite Forge Specialist -> 2D Technical Artist -> Godot Specialist -> QA Lead。
- 默认 UI/UE 路由: 规格设计使用 Producer -> Game Designer -> Art Director -> UI Artist -> UI Programmer -> QA Lead; 效果图和 Figma 落地使用 Art Director -> UI Artist -> UI Programmer -> Godot Specialist -> QA Lead。
- 文档先行: 以后所有玩法、数值、UI、系统或技术结构修改, 都必须先更新对应设计/流程文档, 再实装到游戏中。
- 策划案制图门禁: 玩法和系统流程图可使用 diagrams.net/draw.io、FigJam 等专业工具; 正式 UI/UE 源稿统一使用 Figma/FigJam, 并保留可编辑链接或 .fig 源文件引用。Axure、draw.io、Visio、PPT、Markdown 或 Mermaid 只能作为 UI/UE 草稿、讨论稿或辅助说明, 不能作为最终 UI/UE 源稿。
- 美术审核门禁: 当前游戏视觉方向必须先产出多版效果图和评审文档, 经用户确认后才进入 Sprite Forge、Godot 实装或资产替换。
- 页面美术升级门禁: 升级页面美术时必须锁定 UE; 页面信息、布局、点击目标、控件位置、状态含义、点击反馈节奏完全不变, 只允许升级 2D 视觉皮肤、材质感、描边、阴影、图标、色彩和插画质量。

- UI 工程落地门禁：效果图或 Figma 获批后仍不能直接凭感觉复刻；必须先交付 source reference、design token map、component state matrix、Godot UI plan、响应式/安全区规则、截图对照基线和 UI QA 清单。
- 当前批次实装门禁：当前 1930s 手绘页面批次仍处于效果图评审。用户明确说“实装”前，只允许更新流程文档、评审文档、效果图及其生成工具；不得修改运行时 UI 场景、脚本、资源绑定、输入逻辑或点击区域。
- 简单高品质 2D 门禁：页面美术升级不靠堆复杂度体现品质；优先减少形状、颜色、层级和状态数量，用比例、间距、对比、材质克制、组件复用和小屏可读性提升品质。
- 版本纪律：每次完成修改必须验证、提交 Git，并推送到 GitHub。

1. 开发原则

- 所有任务默认按游戏开发任务处理，采用 Codex Game Studio 的角色视角：制作、玩法、技术、美术/UI、Godot、GDScript、QA。
- 玩法数值优先数据驱动：可配置的设计值进入 `config/tables/`，运行时 JSON 由 `tools/export_config.py` 输出到 `runtime/config/`。
- 配置即时生效：用户或 Codex 每次修改 `config/tables/*.csv` 后，必须在同一轮立刻运行 `tools/validate_config.py` 和 `tools/export_config.py`，让 `runtime/config/*.json` 同步更新；Godot 运行时读取的是导出的 JSON，不直接读取 CSV。
- 美术表现先走专业方向：先做 art brief、参考取舍、视觉方向、可读性检查和多版效果图，再进入资产生成或引擎实现。
- 2D-first 是项目默认：先明确 2D 视角、目标分辨率、sprite/animation 规格、图层/y-sort、atlas、导入设置、碰撞和性能预算；不产出 3D/2.5D 效果图作为默认方向。
- 2D 页面品质默认走简单高品质路线：先锁定当前 UE，再用更少的视觉元素做更清晰的层级；不要新增页面信息、控件、点击状态或反馈节奏来制造“高级感”。
- Sprite Forge 只负责按已批准美术方向执行生成、清理、切图、元数据、预览和 Godot handoff，不替代 Art Director、Visual Development Artist、Concept Artist、Environment Artist 或 UI Artist 的判断。
- 原型表现优先使用引擎内程序化反馈：Tween、缩放、位移、闪烁、粒子、材质调色和 UI 弹跳，避免过早制作序列帧资源。
- UI 先服从已批准的项目视觉方向和目标平台；保留移动端街机所需的短操作路径、强状态反馈和可扫描信息层级，不强制套用固定的粗描边、高饱和或 Web/SaaS 视觉模板。
- 不复制商业游戏的名称、角色、图标、货币、布局、字体或专有资产，只借鉴抽象方向。
- 文档是实现前置条件：先在 `design/` 或 `docs/` 中记录要改的规则、数值、流程、界面或技术决策，再修改配置、脚本、场景和资源。
- 每次完成修改都必须提交 Git，并同步到 GitHub 远端。

2. 标准任务流程

1. 读上游：确认公共流程中对应的角色路由、专项流程和 Definition of Done。
2. 读项目：看 `AGENTS.md`、本文档、相关 `docs/`、当前文件和 `git status`。
3. 定职责：按任务类型使用最小必要角色组合。UI/UE 规格走 `Producer -> Game Designer -> Art Director -> UI Artist -> UI Programmer -> QA Lead`；UI 效果图和 Figma 落地走 `Art Director -> UI Artist -> UI Programmer -> Godot Specialist -> QA Lead`；2D 美术表现走 `Art Director -> Visual Development Artist -> Concept Artist / Environment Artist / UI Artist -> 2D Animation Specialist -> Sprite Forge Specialist -> 2D Technical Artist -> Godot Specialist -> QA Lead`。

4. 文档先行：先更新对应文档。玩法/数值/UI 进入 [design/](#)，工程流程/结构进入 [docs/](#)，必要时同步生成 PDF。
5. 美术预审：涉及视觉方向、资产风格、UI 视觉或宣传级表现时，先输出多版 2D 效果图到 [output/visual_concepts/](#)，并在评审文档中写清取舍、风险和待用户确认点。
6. UE 锁定与品质门槛：页面美术升级必须先写清“不改 UE”的锁定范围；效果图只能表现同一页面信息架构下的皮肤差异，并通过“简单高品质 2D”检查。
7. 专业制图：策划案必须补齐玩法流程图和 UI/UE 图。正式 UI/UE 使用 Figma/FigJam 可编辑源稿，链接或 .fig 引用记录在 [docs/diagrams/](#)；预览图放 [output/diagrams/](#)，PDF 必须嵌入或引用图件。
8. 产品内审：完整页面效果图先由 Producer、Art Director、UI Artist、UI Programmer 和 QA Lead 检查信息完整性、UE 一致性、可读性、位置精度和状态覆盖；P0/P1 问题清零、关键 P2 问题处理后再交用户确认。
9. 用户准入：涉及效果图、UI 皮肤或页面重做时，只有用户明确确认视觉效果并说“实装”，才进入工程交付与运行时修改。
 10. 工程交付：实装前完成 design tokens、组件状态矩阵、Godot UI 计划、响应式/安全区规则、截图基线和 QA 清单。
 11. 定落点：再判断改配置、脚本、场景、资源或工具，避免把设计值写死在代码里。若落点包含 [config/tables/*.csv](#)，必须立即校验并导出 runtime JSON，让配置在本轮开发中生效。
 12. 小步实现：按已经更新且通过评审的文档实装，保持提交范围聚焦，沿用现有脚本、绘制和数据结构。
 13. 本地验证：按改动类型运行对应检查，Godot 脚本改动必须启动项目确认无解析错误；UI 实装必须按目标分辨率生成截图对照。
 14. 整理差异：查看 `git diff --check` 和 `git diff --stat`，确认没有无关破坏。
 15. 提交同步：`git add`、`git commit`、`git push origin main`。
 16. 回报结果：说明先改了哪份文档、实装改了什么、验证了什么、提交号和远端同步状态。

2A. UI/UE 专项流程（公共 v1.9）

2A.1 四段准入

阶段	必须完成	Gate
A. UE 定义与锁定	页面地图、用户路径、线框、信息层级、入口/出口、交互状态、关键反馈、目标分辨率、输入方式和安全区；正式源稿为 Figma/FigJam	页面内容和操作路径可验证；皮肤升级任务已记录“不改 UE”清单
B. 完整页面效果图	使用真实页面信息和当前资源制作整页效果图，不只画背景；覆盖主页面、功能页和弹窗，文本清晰，动态数字有稳定槽位	产品、美术、UI/UE、工程可行性和 QA 内审通过；用户确认效果
C. 工程交付	source reference、design tokens、组件状态矩阵、Godot UI 计划、响应式/安全区规则、截图基线和 UI QA 清单	用户已明确说“实装”；所有交付项可供程序直接执行
D. 实装与验收	可复用 Control/组件、Theme/StyleBox 或等样式资产、输入/focus、动效 hooks、目标分辨率截图和差异记录	UE 不变；截图差异已接受；P0/P1 清零；Godot 与主流程验证通过

2A.2 效果图阶段规则

- 效果图必须是“带完整信息的可评审页面”，不是单独背景、气氛图或只有一个按钮的风格样张。

- 页面信息、布局、点击目标、控件位置、状态含义和反馈节奏以当前 Godot 页面或已批准 Figma/FigJam UE 为唯一基准。
- 皮肤升级不得新增、删除、移动、重命名或重排页面模块、资源栏、导航、按钮和关键反馈。
- 数字和文本必须按最终信息槽位排版。金币、票券、战力等动态数值要在各自数值区视觉居中，并为位数变化预留宽度，不能挤压图标或造成布局跳动。
- 战斗页的核心战斗区域、单元格数量、单元格密度和可操作范围必须保持原版 UE 尺寸；换美术资源不能缩小棋盘或减少格位。
- 所有保留文字必须在目标尺寸可辨认，不得出现乱码、伪字、被边框遮挡或低对比度丢失。
- 若用户指定暂不修改某类素材，例如动物图片，效果图必须沿用该素材，只调整其周边 UI 皮肤和承载关系。
- 页面集必须使用同一套视觉语法、组件比例、边框、材质、色彩、字体方向和图标规则，不能只让首页匹配而其他页面退回旧风格。
- 弹窗也按完整页面处理，至少表现背景遮罩、标题、正文/对象、主次操作、关闭方式、禁用或资源不足状态。
- 当前阶段只输出 docs/、output/visual_concepts/、output/diagrams/ 和必要的效果图生成工具，不写入运行时 assets/、scenes/ 或 scripts/。

2A.3 产品负责人迭代验收

每轮效果图对外提交前，按以下顺序执行内部评审并继续迭代：

1. Producer：检查页面目标、信息完整性、主操作路径、跨页面一致性和用户本轮反馈是否全部落实。
2. Art Director / UI Artist：检查整套视觉身份、层级、材质、图标、字体方向、可读性和原创性。
3. UI Programmer / Godot Specialist：只做可实现性审查，检查组件复用、动态内容槽位、目标分辨率、安全区和状态是否可落地；效果图阶段不得借此提前实装。
4. QA Lead：按标注截图逐项核对，记录 P0/P1/P2。P0/P1 必须清零，影响本轮目标的 P2 必须修正或取得明确接受。
5. 用户评审：提供最新整页效果图和本轮修正摘要；没有用户明确准入就返回效果图迭代，不进入工程阶段。

2A.4 UI 工程交付清单

交付项	本项目要求	建议落点
Source reference	Figma/FigJam 可编辑链接或 .fig 引用，以及获批 PNG/PDF/截图版本号	docs/diagrams/、output/diagrams/、output/visual_concepts/
Design token map	颜色角色、字体层级、间距、圆角、描边、阴影、图标风格、动效时长、安全区	docs/UI_COMPONENT_GUIDE.md
Component state matrix	default、hover、pressed、focused、selected、disabled、locked、loading、success、warning、error、empty、notification、unaffordable	docs/UI_COMPONENT_GUIDE.md
Godot UI plan	可复用 Control、Theme/StyleBox 或等样式资产、signals/events、anchors/containers、focus、输入模式、动效 hooks	对应 UI/技术设计文档
Responsive/safe-area rules	目标宽高比、刘海/圆角/底部手势区、横竖屏策略、文本扩展、触控尺寸和本地化余量	对应 UI/UE 文档
Screenshot parity report	原效果图、Godot 截图、叠图或差异说明；标明一致项和已接受差异	output/ui_qa/<feature>/
UI QA checklist	对比度、触控、focus、换行、数字位数、加载/错误/空状态、减弱动效和无布局跳动	对应 UI/UE 文档

2A.5 Godot UI 落地规则

- 正式页面优先拆成可复用 Control/组件，使用 anchors、containers、Theme/StyleBox、NinePatchRect 或项目等价方案；不得把每页做成互不关联的临时坐标拼图。
- 视觉资产与信息布局分层。背景、边框、装饰不能承担点击逻辑，也不能遮挡文本、数字或点击区域。
- 动态数字使用稳定宽度、对齐和最小/最大位数测试；资源图标与数值分别占位，数值在数值栏居中。
- 移动端必须验证触摸目标、安全区和返回路径；桌面或手柄入口必须验证 hover/focus/pressed 和焦点顺序。
- 动效必须有明确触发、时长、复位和减弱动效策略，不能改变点击目标或造成版面跳动。
- 每个页面至少验证 default、pressed、disabled/locked、loading、empty/error 和资源不足等与该页面相关的非默认状态。
- 实装后必须在项目目标分辨率和至少一个窄屏/宽屏边界分辨率截图，和获批效果图逐项对照。

3. 公共流程阶段映射

公共阶段	本项目状态	本项目执行方式
0. Project Startup	已完成	Git、README、目录、Godot 入口、配置表基础已建立
1. Concept	已有初版	当前方向是移动端卡牌+占地战斗原型，继续在 docs/CURRENT_GAME_DESIGN.md 沉淀
2. Prototype	进行中	用 Godot 快速验证战斗、抽卡、编组、升级、地块和 UI
3. System Design	进行中	规则和数值优先进入文档与配置表，脚本只做运行实现
3A. Data Config	已建立并持续扩展	config/tables/ -> tools/validate_config.py -> tools/export_config.py -> runtime/config/
4. Technical Architecture	需要持续补强	当前集中在 scripts/app/main.gd，复杂度上升后拆分到 scripts/app/ui/、battle/、cards/
5. Vertical Slice	进行中	目标是打通大厅、编组、抽卡、战斗、结算、成长的完整闭环
6. Implementation	按任务推进	每个功能用小步提交推进，保持可运行
7. QA and Tuning	每轮执行	配置校验、GDScript 缩进、Godot 启动、必要时截图/手测
8. Milestone Review	阶段性执行	大功能后判断继续、重做、砍掉或调优
9. Version Finish	每次提交执行	验证通过、commit、push、记录 hash

4. 目录职责

路径	职责	使用规则
config/tables/	设计源表	卡牌、单位、经济、关卡、掉落、地块等可配置内容优先放这里
runtime/config/	运行时配置	由导出工具生成，只有作为引擎读取源时才提交
scripts/	Godot 脚本	放运行时代码、原型逻辑、UI 绘制、配置读取

路径	职责	使用规则
scenes/	Godot 场景	放可运行场景、入口场景和可复用节点
assets/	源资产	美术、音频、UI、卡牌图、建筑图和后续特效资源
tools/	开发工具	校验、导出、文档生成、批处理和 QA 辅助脚本
tests/	自动化测试	放配置、规则、生成器和核心公式测试
docs/	可读文档	设计说明、流程、UI/UX、平衡、结构约定
docs/diagrams/	专业制图源文件与引用	玩法/系统流程图保存专业可编辑源；UI/UE 只记录 Figma/FigJam 可编辑链接或 .fig 源文件引用
output/diagrams/	图件预览	放流程图和 Figma/FigJam 导出的 PNG/SVG/PDF 评审预览，供策划案和 PDF 引用
output/visual_concepts/	当前视觉效果图评审	放多版 AI/概念效果图和风格方向候选；评审通过前不作为最终 runtime 资产
output/ui_qa/	UI 截图验收	UI 获准实装后，按功能保存获批效果图、Godot 截图、叠图/差异说明和验收记录
output/pdf/	审阅版文档	文档 PDF 输出，便于给人查看和归档
tmp/	临时产物	本地临时文件，不提交

5. 改动落点矩阵

需求类型	首选落点	必要验证
卡牌/单位/经济/掉落数值	先改 design/ 或配置说明，再改 config/tables/，随后立即导出 runtime/config/	tools/validate_config.py, tools/export_config.py
战斗规则和交互逻辑	先改 design/，再改 scripts/app/ 或相关 Godot 脚本	GDScript 缩进检查，Godot 启动
UI/UE 规格	先更新 Figma/FigJam 页面地图、用户路径、线框、状态与反馈，并在 docs/diagrams/ 记录可编辑源	可编辑源存在；页面信息、入口/出口、状态和目标分辨率完整
UI 效果图和页面换皮	先锁定 UE，再更新评审文档与 output/visual_concepts/；用户说“实装”前不改 runtime	完整页面与弹窗齐全；信息位置、文字、动态数字槽位、棋盘范围和指定保留素材通过内审与用户确认
UI 实装	用户准入后补齐 design tokens、状态矩阵、Godot UI 计划和截图基线，再改 scripts/app/、scenes/、assets/	Godot 启动；目标/边界分辨率截图对照；状态、输入、安全区、文本和数字稳定性通过 QA
新资源或美术方向	先改美术方向文档，输出多版 2D 效果图并等用户确认，再改 assets/	效果图评审通过，资源能加载，路径不硬编码到错误位置
页面美术升级	先改 UE 锁定美术评审文档，再输出多版 2D 皮肤效果图	页面信息、布局、点击目标、反馈节奏不变；只改美术皮肤，并通过简单高品质 2D 检查
设计决策和流程	docs/	生成 PDF，检查可读性
校验或导出能力	tools/、.github/workflows/	本地运行工具，确认 CI 入口可用

6. 验证门禁

- 任意 .gd 修改后运行 tools/check_gd_indentation.py，并启动 Godot 项目确认没有 parser error。
- 任意配置表修改后必须立即运行 tools/validate_config.py 和 tools/export_config.py；游戏读取 runtime/config/，不允许只改 CSV 却不更新运行时 JSON。

- Godot 在 Windows 出现 应用程序错误、内存不能为 read 或启动即崩溃时，优先检查渲染后端；本项目默认不强制 D3D12，`project.godot` 应使用 Vulkan 作为 Windows 默认渲染驱动，只有在专门兼容性测试通过后才恢复 D3D12。
- `config/tables/` 下的 CSV 必须保持 UTF-8 或 UTF-8 BOM 编码；不要提交 GBK/ANSI 表格。若 `tools/validate_config.py` 报 Unicode decode 错误，先转码源 CSV，再导出 `runtime/config/`。
- 任意文档交付后生成 PDF 到 `output/pdf/`，并渲染检查页面是否可读、无重叠、无截断。
- 任意策划案交付必须检查玩法流程图和 UI/UE 图是否存在专业源文件、预览图和 PDF 可读版本。
- 任意正式 UI/UE 交付必须检查 Figma/FigJam 可编辑链接或 `.fig` 引用；draw.io、Axure、XD、Markdown、Mermaid 和静态 PNG 不能单独充当最终 UI/UE 源稿。
- 任意美术方向交付必须检查 art brief、候选效果图、评审说明、原创性避让点、移动端可读性和用户确认状态；未确认前不得推进到正式资产替换或 Godot 实装。
- 任意页面美术升级必须检查 UE 锁定清单：页面信息不变、控件位置不变、点击目标不变、状态含义不变、点击反馈不变、只替换 2D 视觉风格。
- 任意 2D 页面效果图必须检查简单高品质门槛：少形状、少颜色、少层级、少状态，3 秒内读懂主要信息，UI 覆盖在玩法背景上仍清晰。
- 任意整套 UI 效果图交付必须检查主页面、功能页和弹窗是否使用同一套视觉语言；资源数值是否居中且可容纳位数变化；战斗区和单元格是否保持原 UE；所有保留文字是否清晰；用户要求暂不修改的素材是否原样保留。
- 任意 UI 实装前必须有用户明确准入，以及 source reference、design tokens、组件状态矩阵、Godot UI 计划、响应式/安全区规则、截图基线和 UI QA 清单。
- 任意 UI 实装后必须在项目目标分辨率和边界分辨率生成截图对照，检查文本溢出、触控/focus、动态数字跳动、loading/empty/error/locked/disabled/pressed/资源不足状态和减弱动效。
- 提交前运行 `git diff --check`，确认没有尾随空白或格式错误。
- 推送前确认 `git status --short --branch`，避免遗漏文件。

7. Godot 与 GDScript 约定

- 本项目 `.gd` 使用 tab 缩进，规则写在 `.editorconfig`。
- 不做盲目的 tab/space 全局替换；修复缩进时先保护代码块层级。
- 运行错误优先从 Godot parser/debug 输出定位，再修代码。
- UI 原型当前以 `scripts/app/main.gd` 的绘制函数为主，新增抽象时必须避免让单文件继续无边界膨胀；可复用逻辑成熟后再拆分模块。

8. Git 与 GitHub 同步

- 工作区可能已有他人或前序未提交改动；先确认来源和范围，不回滚无关修改。
- 提交保持聚焦，但如果用户要求“每次修改都同步”，完成后必须提交并推送。
- 默认远端分支为 `main`，推送命令为 `git push origin main`。
- 每次最终回报包含提交号、验证结果和工作区是否干净。

9. 专业美术表现流程

本项目从公共流程 v1.6 起按 2D-first 专业美术生产线推进；自公共流程 v1.9 起，UI/UE 再增加 Figma/FigJam 正式源稿与效果图工程交付门禁。“简单高品质 2D”仍是页面美术和效果图的默认质量门槛。任何会影响游戏视觉身份、资产风格、UI 视觉、关键画面、角色/场景/道具/地图/FX 质量的任务，都必须先完成方向确认，再进入资产生成或 Godot 实装。

默认路由：

Producer -> Creative Director -> Art Director -> Visual Development Artist -> Concept Artist / Environment Artist / UI Artist -> 2D Animation Specialist -> Sprite Forge Specialist -> 2D Technical Artist -> Godot Specialist -> QA Lead

本项目执行顺序：

1. Art brief：说明目标玩家、平台、镜头、玩法读图需求、性能约束和商业参考避让点。
2. Visual direction：整理风格关键词、形状语言、色彩策略、材质策略、光照和氛围原则。
3. 多版 2D 效果图：至少提供 3 个可比较方向；当前游戏默认存放在 [output/visual_concepts/](#)。整体视觉方向文档为 [docs/CURRENT_GAME_VISUAL_CONCEPT_OPTIONS.md](#)，UE 锁定页面皮肤文档为 [docs/CURRENT_GAME_2D_UE_LOCKED_ART_OPTIONS.md](#)。页面皮肤必须以少形状、少颜色、少层级、少状态为起点，用比例、间距、对比、材质克制和组件复用体现品质。
4. 用户评审：用户明确选择、混合或否定方向前，不推进到正式资产替换、UI 重绘或场景落地。
5. 生产设定：通过后再产出角色、场景、UI、图标、地图或 FX 的 production sheet、尺寸、状态、pivot、anchor、碰撞提示和命名规则。
6. Sprite Forge / 引擎执行：只按已批准方向生成、清理、切片、元数据、预览和 Godot handoff。
7. QA：检查小尺寸可读性、主体/背景分离、UI 层级、色盲风险、原创性、资源加载和性能风险。

当前审核节点：

- 已为当前游戏输出多版效果图到 [output/visual_concepts/](#)。
- 评审文档：[docs/CURRENT_GAME_VISUAL_CONCEPT_OPTIONS.md](#)。
- 当前 1930s 页面批次的专项评审文档为 [docs/CURRENT_GAME_1930S_PAGE_STYLE_OPTIONS.md](#)。
- 在用户明确确认方向并说“实装”前，只允许继续补充说明、内部验收或追加候选，不进入 Godot 实装和资产替换。

页面美术升级的额外约束：

- 必须使用现有 UI/UE 图和当前 Godot 页面结构作为锁定基准。
- 不改变信息架构、控件数量、控件位置、点击区域、状态含义和反馈节奏。
- 效果图只比较 2D 视觉皮肤：面板、边框、按钮材质、色彩、图标风格、地块质感、光影层级和整体品质。
- 皮肤差异必须足够克制：不新增装饰性页面模块，不增加额外点击状态，不让材质、描边或背景抢走棋盘/CTA/资源条的可读性。

10. 后续结构演进方向

- 当 [scripts/app/main.gd](#) 继续增长时，优先拆出 [scripts/app/ui/](#)、[scripts/app/battle/](#)、[scripts/app/cards/](#)、[scripts/app/config/](#)。

- 将硬编码原型参数逐步迁移到 [config/tables/](#)，保留脚本中的常量只作为临时桥接。
- 为核心规则补 [tests/](#)：卡牌升级、抽卡概率、地块生成、战斗结算和配置导出。
- UI 获准实装时必须建立或完善 [docs/UI_COMPONENT_GUIDE.md](#)，把卡牌、资源条、地块提示、底部导航、弹窗及其状态做成可复用规范，并记录 design tokens 与组件状态矩阵。

11. 流程同步记录

日期	上游版本	项目同步结论
2026-07-10	v1.9	已确认并同步 UI/UE 正式源稿、效果图工程交付、产品负责人迭代验收、用户实装准入、Godot UI 计划、响应式/安全区、截图对照和 UI QA Gate；当前 1930s 页面批次继续只做效果图，不实装